Which Functors Are Containers?

A chapter of *The Category of Containers*

MacBeth

Abstract

Hand me a functor $F\colon \mathbf{Set} \to \mathbf{Set}$. Is it a container? This chapter answers the question exactly. First we see that a container is *literally* a family of sets, so that **Cont** is the free coproduct completion $\mathbf{Fam}(\mathbf{Set}^{\mathrm{op}})$ — a fact that makes everything else fall out. Then comes the test: F is a container if and only if it *preserves connected limits*. We compute the test on friendly functors, watch the covariant powerset fail it by a one-line counting argument, and then recover a container's positions from its functor. Along the way we correct the formula everyone reaches for first — the fibre of $F(2) \to F(1)$ — which computes the *powerset* of the positions, not the positions themselves. We close with limits and colimits in **Cont**: it is complete and cocomplete, the extension functor $[\![-]\!]$ preserves connected limits and coproducts, and it stops respecting colimits at exactly the place where quotients manufacture symmetries and carry an object out of the polynomial world. Everything substantive here is classical — Diers, Carboni–Johnstone, Gambino–Kock; our contribution is arrangement, the concrete non-examples, and the honest correction. The products and coproducts are machine-checked.

The question

A container $(S \triangleleft P)$ is a set S of shapes together with, for each shape s, a set $P(s)$ of positions; its extension is the functor

$$[\![(S \triangleleft P)]\!](X) \;=\; \coprod_{s \in S} X^{P(s)} \;=\; \big\{(s, v) \mid s \in S,\; v\colon P(s) \to X\big\}.$$

We met this in the opening chapter, and we proved the representation theorem: the extension functor $[\![-]\!]\colon \mathbf{Cont} \to [\mathbf{Set}, \mathbf{Set}]$ is full and faithful, so the combinatorics of containers captures exactly the natural transformations between their extensions [Cited: AAG05].

Full and faithful tells us that $[\![-]\!]$ is an isomorphism *onto its image*. It does not tell us what that image is. So the question that organises this chapter is the obvious next one, and it is sharper than it looks:

Given a functor $F\colon \mathbf{Set} \to \mathbf{Set}$, is it a container?

This is not idle taxonomy. The whole programme of this book — the equivalence chain, the comonoids, the monoidal structures — lives *inside* the image of $[\![-]\!]$. Everything we can say about containers is a statement about functors of this special shape. Knowing which functors are of this shape is knowing the exact reach of the theory: what it covers, and — just as important — where its boundary runs and what lies on the far side.

The promise. By the end of this chapter you will be able to look at a functor and decide whether it is a container; you will know how to read its shapes and positions back off; and you will see why the first formula you would guess for the positions is wrong. The tools are a single structural identification and a single theorem, and both are best met after a little computation.

1 A container is a family of sets

Before we test functors, we should see **Cont** for what it is. Strip the extension away and look at the raw data of a container: a set S, and for each $s \in S$ a set $P(s)$. That is an S-indexed family of sets. Nothing more.

Families of objects of a category $\mathcal{C}$ are themselves the objects of a category, the *free coproduct completion* $\mathbf{Fam}(\mathcal{C})$. Its objects are pairs $(S, P\colon S \to \mathcal{C})$; a morphism $(S,P) \to (T,Q)$ is a reindexing function $u\colon S \to T$ together with, for each $s \in S$, a $\mathcal{C}$-morphism comparing $P(s)$ with $Q(u(s))$. The completion turns coproducts that $\mathcal{C}$ may lack into formal ones: a family (S,P) *is* the formal coproduct $\coprod_{s\in S} P(s)$.

Now watch the variance. A container morphism $(S \triangleleft P) \to (T \triangleleft Q)$, as we defined it, is a shape map $u\colon S \to T$ *forward* together with position maps $f_s\colon Q(u(s)) \to P(s)$ running *backward*. Backward is exactly a morphism in $\mathbf{Set}^{\mathrm{op}}$. So the comparison lives in $\mathbf{Set}^{\mathrm{op}}$, not $\mathbf{Set}$, and we have identified the category on the nose.

Proposition 1.1 (**Cont** is the free coproduct completion of $\mathbf{Set}^{\mathrm{op}}$)**.** *There is an isomorphism of categories* $\mathbf{Cont} \cong \mathbf{Fam}(\mathbf{Set}^{\mathrm{op}})$*, the identity on objects, sending a container morphism* (u, f) *to the reindexing* u *together with the family* $(f_s)_{s\in S}$ *read as morphisms in* $\mathbf{Set}^{\mathrm{op}}$*.* *[Folklore]*

This is not deep, but it is clarifying, and it pays for itself immediately. Chapter 0 showed that the free coproduct completion of any category $\mathcal{C}$ embeds into $[\mathcal{C}^{\mathrm{op}}, \mathbf{Set}]$ by the left Kan extension of the Yoneda embedding along itself — concretely, a family becomes a coproduct of representables. Feed it $\mathcal{C} = \mathbf{Set}^{\mathrm{op}}$ and the abstract statement becomes our extension functor:

$$(S,P) \;\longmapsto\; \coprod_{s\in S} y^{P(s)}, \qquad y^A = \mathrm{Hom}_{\mathbf{Set}}(A, -),$$

because a representable on $\mathbf{Set}^{\mathrm{op}}$ evaluated in $\mathbf{Set}$ is the covariant homfunctor $y^A = (-)^A$. So $[\![(S \triangleleft P)]\!] = \coprod_s y^{P(s)}$ is not a definition we imposed; it is what the free coproduct completion of $\mathbf{Set}^{\mathrm{op}}$ *does* when you land it in functors. Containers are exactly coproducts of representable endofunctors of **Set**. Hold that phrase — *coproduct of representables* — because it is the whole answer to our question, once we understand which functors it describes.

Terminology hazard: positions versus directions

The reader who also reads Spivak's *Poly* must keep two dictionaries. There, a polynomial functor is written $p = \sum_{i\in p(1)} y^{p[i]}$, the elements of $p(1)$ are called *positions*, and the elements of each fibre $p[i]$ are called *directions*. In our language the elements of $p(1) = S$ are *shapes* and the elements of the fibres $P(s)$ are *positions*. The naming is exactly opposite: Spivak's "positions" are our shapes, and his "directions" are our positions. The objects are the same; only the words differ. We keep shapes-and-positions throughout, because in the applications a shape is a structure and a position is a slot inside it, and that is the intuition we want the words to carry.

2 The test

We know the target shape: a container extension is a coproduct of representables, $F(X) \cong \coprod_{s\in S} X^{P(s)}$. So the question "is F a container?" is the question "can F be written that way?" We gather evidence by computing: the arithmetic of finite sets decides several cases outright, and — better — teaches the method.

Here is the fact that does the work. If $F = [\![(S \triangleleft P)]\!]$ and X is a finite set with $|X| = n$, then

$$|F(n)| \;=\; \sum_{s \in S} n^{\,|P(s)|}.$$

The cardinality of a container extension at n is a sum of powers of n, one power per shape, the exponent being the number of positions. Evaluations at small n probe this sum. At $n = 1$ every power is 1, so $|F(1)| = |S|$: the value at a one-element set counts the shapes. At $n = 0$ every positive power vanishes and $0^0 = 1$, so $|F(0)|$ counts the shapes with *no* positions — the constants. These two readings alone are often enough.

Example 2.1 (Friendly functors pass)**.** $F(X) = 1 + X$: shapes $\{0, 1\}$ with $|P| = 0, 1$; extension $\coprod\{X^0, X^1\}$. A container. $F(X) = X^2$: one shape, two positions. A container. Lists, $F(X) = \coprod_n X^n$: shape $n \in \mathbb{N}$, positions $\{0, \ldots, n-1\}$. A container. In each case the coproduct-of-powers form is visible on the page.

Example 2.2 (The covariant powerset fails — and the argument is one line)**.** Let $\mathcal{P}\colon \mathbf{Set} \to \mathbf{Set}$ be the covariant powerset functor, $\mathcal{P}(X) = \{\text{subsets of } X\}$, acting on maps by direct image. Is it a container?

Count. $\mathcal{P}(\varnothing) = \{\varnothing\}$ has one element; $\mathcal{P}(1) = \{\varnothing, \{*\}\}$ has two; $\mathcal{P}(2)$ has four; in general $|\mathcal{P}(n)| = 2^n$. Suppose, for contradiction, $\mathcal{P} = [\![(S \triangleleft P)]\!]$. From $n = 1$ we read $|S| = |\mathcal{P}(1)| = 2$: two shapes. From $n = 0$ we read that exactly $|\mathcal{P}(0)| = 1$ of them has no positions; call the other shape $s_\star$, with $|P(s_\star)| = p$ positions. Now evaluate at $n = 2$:

$$4 \;=\; |\mathcal{P}(2)| \;=\; \sum_{s \in S} 2^{|P(s)|} \;=\; 2^0 + 2^p \;=\; 1 + 2^p.$$

So $2^p = 3$, which no natural number p satisfies. The covariant powerset is *not* a container.

The powerset example is worth pausing on, because the reason it fails is the reason the theory has a boundary at all. A subset of X is a collection of elements with no record of *where* each element sits. A container labelling $v\colon P(s) \to X$, by contrast, assigns each position its own element, and the position is a name that persists. Powerset forgets the names; it is a functor built by *quotienting*, and quotients are exactly what containers cannot do. We will meet this boundary again in §4, from the colimit side, and it will be the same wall.

Counting settles individual cases, but we want a criterion that works for every functor, finite or not. Here it is. The right notion is that of a *connected limit*: a limit whose indexing diagram is connected (any two objects joined by a zig-zag of arrows). A pullback is connected — its shape $\bullet \to \bullet \leftarrow \bullet$ is joined up — and so are wide pullbacks (many arrows into one target) and cofiltered limits. A product is *not*: its shape is a bare set of objects with no arrows between them, so it falls into disconnected pieces. Connected versus disconnected is exactly the line the characterisation runs along.

Theorem 2.3 (Characterisation of containers)**.** *For a functor $F\colon \mathbf{Set} \to \mathbf{Set}$ the following are equivalent.*

(i) *F is a container extension: $F \cong [\![(S \triangleleft P)]\!]$ for some container.*

(ii) *F preserves connected limits (equivalently: wide pullbacks and cofiltered limits).*

(iii) *F is a coproduct of representable functors.*

[Cited: Gambino–Kock 2009, §1.18 & Prop. 1.22; the equivalence is due to Diers 1977 and Carboni–Johnstone 1995]

Why is preserving connected limits the same as being a coproduct of representables? The intuition is short and worth carrying. A single representable $y^A = (-)^A$ preserves *all* limits: it is the right adjoint to $- \times A$, and right adjoints preserve limits. The question is what survives when we glue representables by a coproduct. In **Set**, coproducts commute with connected limits but not with arbitrary ones: a connected diagram cannot "see" the disjoint branches of a coproduct separately, so the coproduct passes through the limit, whereas a product (disconnected) can pair up elements across different branches and breaks the interchange. Preserving connected limits is therefore precisely the trace left in $[\mathbf{Set}, \mathbf{Set}]$ by "being a coproduct of things that preserve everything." Condition (ii) is the shadow of condition (iii).

Why "connected", and neither "wide pullbacks" nor "all"

Condition (ii) is easy to mis-state in either direction.

Not merely wide pullbacks. Wide pullbacks are the connected limits one meets most often, so one is tempted to shorten (ii) to "preserves wide pullbacks". That is genuinely weaker: cofiltered limits are connected too, yet they are not built from pullbacks, and they must be demanded separately. "Preserves connected limits" unpacks to wide pullbacks *and* cofiltered limits together, and dropping the latter loses functors [Cited: Gambino–Kock, §1.18].

Not all limits either. A container need *not* preserve the *empty* limit — the terminal object — because $[\![(S \triangleleft P)]\!](1) = S$ can be any set, whereas preserving the empty limit would force $F(1) \cong 1$. The empty diagram is *disconnected*, so it sits rightly outside (ii). Connectedness is exactly the dividing line: it keeps the cofiltered limits, which containers respect, and discards the products and the empty limit, which they do not.

3 Reading the positions back off

The test tells us *whether* F is a container. Suppose it passes. How do we recover the data $(S \triangleleft P)$?

The shapes are free: $S = F(1)$, exactly as the counting in §2 used. A one-element set has one labelling per shape, so $F(1) = \coprod_s 1^{P(s)} = \coprod_s 1 = S$. That half is honest arithmetic. The positions are the subtle half, and here is where a natural first guess goes wrong.

Correction: the fibre of $F(2) \to F(1)$ is *not* $P(s)$

The tempting formula runs: the unique map $2 \to 1$ induces $F(2) \to F(1) = S$; take the fibre over a shape s and call it the positions. Compute what this actually returns. Since $F(2) = \coprod_{s'} 2^{P(s')}$ and the map to S forgets the labelling $P(s') \to 2$ and remembers only s', the fibre over s is

$$\left\{ \varphi \colon P(s) \to 2 \right\} \;=\; 2^{P(s)},$$

the set of *subsets* of $P(s)$ — the powerset of the positions, not the positions. Evaluation-and-fibre overcounts by an exponential. No single evaluation of F, followed by a fibre, can extract $P(s)$: the functor at a fixed set only ever sees *labellings* of the positions, never the positions bare.

The correction is not a patch; it is a signpost pointing at the real mechanism. To see $P(s)$ itself we must ask for the labelling that names each position *by itself* — the identity labelling — and that is a universal property, not an evaluation.

Proposition 3.1 (Position recovery via the generic element)**.** *Let F be a container. For each shape $s \in F(1)$ there is a set $P(s)$ and a* generic element *$g_s \in F(P(s))$ lying over s, universal in the sense that for every set X and every $x \in F(X)$ lying over s there is a unique map $\alpha\colon P(s) \to X$ with $F(\alpha)(g_s) = x$. This $P(s)$ is the object of positions, and*

$$F(X) \;\cong\; \coprod_{s \in F(1)} X^{P(s)}$$

is the container form, recovered componentwise by the Yoneda lemma. The generic element is the initial object of the category of elements of F sitting over s; its existence is exactly the connected-limit preservation of Theorem 2.3. *[Cited: Gambino–Kock 2009, Prop. 1.22]*

The generic element is the identity labelling $g_s = (s, \mathrm{id}_{P(s)})$ wearing abstract clothing: it is the one element of $F(P(s))$ from which every other labelled structure of shape s is obtained by relabelling, and by exactly one relabelling. "Exactly one" is the universal property, and it is what pins $P(s)$ down to the positions rather than to any of their exponential shadows.

Remark 3.2 (The derivative sees the total position set)**.** A second, tidier-looking phrasing is worth knowing — and worth not overtrusting. The total space of positions $\coprod_s P(s)$ is the value at 1 of the *derivative* ∂F — the container of one-hole contexts developed in the chapter on the chain rule — and $P(s)$ is its fibre over s. This is correct, but it is a restatement, not a shortcut: ∂F is itself a derived construction, not a single evaluation of F, so it does not escape the moral of the correction box. If you want the positions, you must, one way or another, invoke the universal property.

4 Limits and colimits in Cont

We have been studying $[\![-]\!]$ as a bridge from **Cont** into $[\mathbf{Set}, \mathbf{Set}]$. It is natural to ask how much categorical structure the bridge carries across. The identification of §1 answers the first half at a stroke: free coproduct completions are complete and cocomplete, so **Cont** *is both.* What is delicate is which limits and colimits $[\![-]\!]$ *preserves*, and the characterisation theorem has already told us where the answer turns.

Start with the structure that is easy, concrete, and machine-checked. Binary products and coproducts in **Cont** have a clean description, and the variance is the whole lesson.

Proposition 4.1 (Products and coproducts of containers)**.** *In* **Cont***:*

- *the* coproduct *of $(S \triangleleft P)$ and $(T \triangleleft Q)$ has shapes $S + T$ and positions inherited case-wise (P on the left summand, Q on the right);*
- *the* product *of $(S \triangleleft P)$ and $(T \triangleleft Q)$ has shapes $S \times T$ and, over (s, t), positions $P(s) + Q(t)$ — a* coproduct *of position sets.*

[MacBeth, Lean-verified: Cont.lean]

The product deserves a second look, because it inverts the naive expectation. The positions of a product are a *sum*, not a product. The reason is the variance of §1 playing out on universal properties: the projection $\pi_1\colon (S \triangleleft P) \times (T \triangleleft Q) \to (S \triangleleft P)$ has the forward shape map $(s, t) \mapsto s$, and on positions it must run *backward*, so it is the coproduct injection $P(s) \hookrightarrow P(s) + Q(t)$. A morphism into the product is a pair of morphisms, and pairing forward shape maps forces summing backward position maps. Under $[\![-]\!]$ this reads as the familiar polynomial identity $[\![(S \triangleleft P) \times (T \triangleleft Q)]\!](X) =$

$\coprod_{s,t} X^{P(s)+Q(t)} = \coprod_{s,t} X^{P(s)} \times X^{Q(t)}$, the product of the two extensions — so $[\![-]\!]$ preserves this product. It is worth being clear about why that is a separate fact, and not a corollary of the characterisation theorem.

What $[\![-]\!]$ preserves, and where it stops

Theorem 2.3 says a container extension *preserves connected limits*. Products are the archetypal *dis*connected limit, so connected-limit preservation says nothing about them directly; nevertheless $[\![-]\!]$ does preserve products, by the explicit computation above (and coproducts, since a coproduct of shapes maps to a coproduct of extensions). The clean summary is:

$$[\![-]\!] \text{ preserves } \begin{cases} \text{connected limits} & \text{(Theorem 2.3)} \\ \text{products and coproducts} & \text{(Proposition 4.1)} \end{cases}$$

but $[\![-]\!]$ is *not* cocontinuous: it does not preserve general colimits. In particular it does not preserve coequalisers.

Why should coequalisers be where preservation breaks? Because a coequaliser is a quotient, and we have already watched a quotient carry an object out of the container world once. The covariant powerset of §2 is itself a quotient of a coproduct of representables: it is the list functor with its labellings identified whenever they differ by a rearrangement or a repetition. That double identification is exactly what a coproduct of representables cannot express — a representable names its positions and keeps them distinct — and it is why the counting failed. Coequalisers reproduce the phenomenon in general: coequalise two container morphisms and the resulting functor can acquire automorphisms of its positions. The mildest such quotients, by a group acting on positions, are the *analytic* functors of Joyal's species, which are already outside the image of $[\![-]\!]$; powerset lies further out still, its idempotent identification being coarser than any group action. Either way the lesson is one line: the coproduct-of-representables form is closed under connected limits and under coproducts, and not under quotients.

Open thread for a later pass

The precise statement "$[\![-]\!]$ does not preserve coequalisers" deserves a pinned reference and a concrete two-container coequaliser exhibiting the failure. Abbott's thesis is the place the author expects to find it; that citation is *not* yet verified against the source, and no specific theorem number should be attributed until it is. Treat this paragraph's claim as the honest folklore it is — consistent with everything above — and see the citation tracker.

5 The boundary of the theory

Step back and the chapter's three findings line up into one picture. A container is a family of sets; $\mathbf{Cont} = \mathbf{Fam}(\mathbf{Set}^{\mathrm{op}})$; and its extension is a coproduct of representables. Being a coproduct of representables is a property a functor can be tested for — preservation of connected limits — and when it holds, the shapes are $F(1)$ and the positions are named by generic elements, not by fibres. And the same form that these coproducts take is what fails under quotients: powerset from the value side, coequalisers from the colimit side. Both failures are one failure. Containers are what you get by *summing* representables; you leave them the moment you *quotient*.

That boundary is the polynomial boundary the book keeps returning to. Inside it live the objects on which the whole equivalence chain runs — directed containers, comonoids, small categories,

the monoidal structures of the later chapters — and each of those structures will silently use that its underlying functor is a coproduct of representables, that its shapes are recoverable, that connected limits pass through. Outside it lie the analytic and symmetric functors, the probability and powerset monads, the quotients: a rich world, but one where the equivalence chain fractures, and where compositional correctness has to be re-earned rather than inherited. Knowing which functors are containers is knowing where you are standing.

The next chapter puts the boundary to work: it asks which containers carry a *comonoid* structure, and finds — the central surprise of the book — that the answer is small categories.

Provenance summary

- **Cont** $\cong$ **Fam**(**Set**$^{\mathrm{op}}$) (Prop. 1.1): folklore; the free coproduct completion, embedded via the Kan extension of Chapter 0.
- **Characterisation and recovery** (Thm. 2.3, Prop. 3.1): cited to Gambino–Kock, *Polynomial functors and polynomial monads*, arXiv:0906.4931, §1.18 and Prop. 1.22, which report the equivalence as due to Diers (1977) and Carboni–Johnstone (*MSCS* 5, 1995). Not claimed as new; exposition and the non-examples are ours.
- **Representation theorem** (used, not reproved): Abbott–Altenkirch–Ghani, *TCS* 342 (2005); the book's earlier chapter.
- **Products and coproducts** (Prop. 4.1): MacBeth, machine-checked in `Cont.lean` (the `lean/` tree).
- **Non-cocontinuity / coequaliser failure** (§4): stated as folklore; the exact reference (Abbott's thesis) is flagged as not yet pinned.

References

[1] M. Abbott, T. Altenkirch, N. Ghani. *Containers: constructing strictly positive types.* Theoretical Computer Science **342** (2005), 3–27.

[2] N. Gambino, J. Kock. *Polynomial functors and polynomial monads.* arXiv:0906.4931; Math. Proc. Cambridge Philos. Soc. **154** (2013), 153–192.

[3] A. Carboni, P. Johnstone. *Connected limits, familial representability and Artin glueing.* Mathematical Structures in Computer Science **5** (1995), 441–459. (Corrigenda: *MSCS* **14** (2004), 185–187.)

[4] Y. Diers. *Catégories localisables.* Thèse de doctorat, Université Paris VI, 1977.